

Lab Practical 03

C#, .NET Fundamentals, ASP.NET Core & REST API Development

Lab Practical Number	Lab 03
Lab Title	C#, .NET Fundamentals, ASP.NET Core & REST API Development
Related Lecture	Lecture 03 – C#, .NET Fundamentals, ASP.NET Core and REST API Development
Duration	2 Hours (in-lab)
Learning Outcomes	LO2 – Apply frameworks & tools • LO3 – Best practices for integration & quality • LO4 – Select & justify approaches
Stack	C# 12 • .NET 8 (LTS) • ASP.NET Core Web API • Swagger / Postman
Mode	Individual coding • Word document + project ZIP submitted to CourseWeb within the session

Introduction

This lab turns Lecture 03 into a working backend. In the lecture you studied C# and object-oriented programming, the .NET platform and ASP.NET Core, REST API architecture (resources, HTTP methods, status codes, JSON), and the layered controller–service–model structure. In this 2-hour session you will build a small but complete ASP.NET Core REST API with full CRUD operations and test it in Swagger.

Everything you need comes from Lecture 03. Base your implementation and explanations on the lecture content, the worked LankaMart example, and the class examples (decimal for money, models vs DTOs, dependency injection, attribute routing, correct status codes). You may add small examples of your own, but stay within the scope of Lecture 03.

CONTINUITY WITH LAB 02

In Lab 02 you built the CampusEats React frontend that fetched a menu. In this lab you build the ASP.NET Core REST API that serves that menu — the GET/POST/PUT/DELETE endpoints behind those components.

Lab Objectives

By the end of this lab, you should be able to:

- **Create and run an ASP.NET Core Web API project** and navigate its layered folder structure (LO2, LO3).
- **Write a C# model and DTOs** using correct types (decimal for money) and understand why the public DTO differs from the internal model (LO2, LO3).
- **Build a service with an interface and register it for dependency injection** to hold business logic and in-memory data (LO2, LO3).
- **Implement a REST controller with GET, POST, PUT and DELETE** using attribute routing and correct HTTP status codes (LO2, LO3).
- **Test every endpoint in Swagger (or Postman)** and confirm the right status code for each operation (LO3).

- **Justify ASP.NET Core and REST design decisions** for the backend using the lecture's method and status-code rules (LO4).

Required Tools / Software

- **.NET 8 SDK** — verify with `dotnet --version` (should report 8.x).
- **Visual Studio 2022, Rider, or VS Code** — with the C# Dev Kit extension for VS Code.
- **A web browser** — to use the built-in Swagger UI at `/swagger`.
- **Postman (optional)** — for saved request collections and scripted assertions, as shown in the lecture.
- **A terminal** — to run the dotnet CLI commands.

TECHNICAL NOTE

This lab targets **.NET 8**, whose Web API template ships with Swagger UI. If your machine has .NET 9 or 10 and Swagger does not appear, see the troubleshooting note in the Lecturer Notes. Use the `--use-controllers` template flag so you get a controller-based project, not a minimal API.

Lab Time Plan (120 minutes)

Time	Activity	Output
0:00 – 0:10	Briefing & recap of Lecture 03 (OOP, DI, REST, layers)	Ready to code
0:10 – 0:22	Task 01 — Create the Web API project + run Swagger	Running API + Swagger
0:22 – 0:40	Task 02 — Model + DTOs + validation	Model & DTO classes
0:40 – 1:05	Task 03 — Service (interface + logic) + register for DI	Service with seed data
1:05 – 1:30	Task 04 — Controller with GET + POST, tested in Swagger	GET & POST working
1:30 – 1:50	Task 05 — PUT + DELETE, full CRUD test + LO4 justification	Full CRUD API
1:50 – 2:00	Quick test + screenshots, zip and submit	Submitted .docx + .zip

HOW TO USE THIS PLAN

These times are a guide. Get the project running and the model/DTOs in place quickly, then protect time for the service and controller (Tasks 03–04), which carry the most marks. Add PUT/DELETE last.

Mini Project Scenario

“CAMPUSEATS” — MENU API (PRODUCT/INVENTORY STYLE)

You are building the **backend REST API** for CampusEats. It manages a menu of dishes — each dish is a resource with a name, price, category, and availability. The API must support listing, viewing one, creating, updating, and deleting dishes.

Data is held **in memory** (a list inside the service) so you can focus on C#, REST design, and the controller–service structure. Persisting to a database with EF Core comes in the next lecture.

The resource

Field	Type & notes
Id	int — assigned by the server (not supplied on create).
Name	string — required, the dish name.
Price	decimal — money is always decimal, never double.
Category	string — e.g. Mains, Snacks, Drinks, Dessert.
Available	bool — whether the dish can currently be ordered.

Endpoints you will build

Method	Path	Success	Purpose
GET	/api/menu	200	List all dishes
GET	/api/menu/{id}	200 / 404	Get one dish
POST	/api/menu	201	Create a dish
PUT	/api/menu/{id}	204 / 404	Replace a dish
DELETE	/api/menu/{id}	204 / 404	Delete a dish

Target project structure (Lecture 03, Part 5)

```
CampusEats.Api/
  Program.cs           // startup, DI, middleware
  appsettings.json     // configuration (no secrets!)
  Controllers/
    MenuController.cs  // HTTP layer (thin)
  Services/
    IMenuService.cs    // contract (interface)
    MenuService.cs     // business logic + in-memory data
  Models/
    MenuItem.cs        // domain entity
  Dtos/
    CreateMenuItemDto.cs // input shape
    MenuItemDto.cs     // output shape
```

TASK 01 • 12 minutes • LO2, LO3**Create the Web API Project & Run Swagger**

Scaffold a controller-based ASP.NET Core Web API, run it, and open the Swagger UI. Then explore the layered folder structure from the lecture.

Step-by-step instructions

1. In a terminal, create a **controller-based** Web API and run it:

Terminal

```
$ dotnet new webapi --use-controllers -n CampusEats.Api
$ cd CampusEats.Api
$ dotnet run
```

2. Open the printed HTTPS URL and add `/swagger` (for example `https://localhost:7xxx/swagger`). You should see the interactive API docs.
3. If prompted about the HTTPS certificate, trust it once with `dotnet dev-certs https --trust`.
4. Create the folders `Models`, `Dtos`, and `Services` inside the project (you will fill them next).
5. Delete the sample `WeatherForecast.cs` and `Controllers/WeatherForecastController.cs` so you start clean.

EXPECTED OUTPUT

The API runs and Swagger UI loads in the browser. Take a screenshot of the running Swagger page and of your project's folder structure.

TASK 02 • 18 minutes • LO2, LO3

Create the Model & DTOs

Create the internal domain model and the public DTOs. Remember the lecture's rule: the **model** is the internal shape, while a **DTO** (Data Transfer Object) is the public contract that crosses the API boundary.

Step-by-step instructions

1. Create the model `Models/MenuItem.cs` — note `decimal` for the price:

```
src: Models/MenuItem.cs
namespace CampusEats.Api.Models;

public class MenuItem
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; } // money = decimal
    public string Category { get; set; } = "";
    public bool Available { get; set; } = true;
}
```

2. Create the DTOs in `Dtos/`. Use `record` types and add validation attributes on the input DTO (Lecture 03, Part 6):

```
src: Dtos/CreateMenuItemDto.cs & MenuItemDto.cs
using System.ComponentModel.DataAnnotations;

namespace CampusEats.Api.Dtos;

// input shape — no Id (the server assigns it)
public record CreateMenuItemDto(
    [Required, StringLength(80)] string Name,
    [Range(0, 100000)] decimal Price,
    [Required, StringLength(40)] string Category);

// output shape — the public contract
public record MenuItemDto(
    int Id, string Name, decimal Price,
    string Category, bool Available);
```

3. In your write-up, explain in one or two sentences why the create DTO has no `Id` and why a model and a DTO are kept separate.

EXPECTED OUTPUT

The `MenuItem` model and the two DTO records compile. A short explanation of model-vs-DTO and why the input DTO omits `Id`.

TASK 03 • 25 minutes • LO2, LO3**Create the Service & Register it for DI**

Put the business logic and in-memory data in a service behind an interface. The controller will depend on the **interface**, and the DI container will supply the implementation — exactly as the lecture describes.

Step-by-step instructions**1. Create the contract `Services/IMenuService.cs`:**

```
src: Services/IMenuService.cs
using CampusEats.Api.Dtos;

namespace CampusEats.Api.Services;

public interface IMenuService
{
    IEnumerable<MenuItemDto> GetAll();
    MenuItemDto? GetById(int id);
    MenuItemDto Create(CreateMenuItemDto dto);
    bool Update(int id, CreateMenuItemDto dto);
    bool Delete(int id);
}
```

2. Create the implementation `Services/MenuService.cs`. Part 1 — seed data and the read methods:

```
src: Services/MenuService.cs (part 1)
using CampusEats.Api.Dtos;
using CampusEats.Api.Models;

namespace CampusEats.Api.Services;

public class MenuService : IMenuService
{
    private readonly List<MenuItem> _items;
    private int _nextId;

    public MenuService() // seed in-memory data
    {
        _items = new()
        {
            new() { Id = 1, Name = "Kottu Roti",
                    Price = 750m, Category = "Mains" },
            new() { Id = 2, Name = "Fried Rice",
                    Price = 850m, Category = "Mains" },
            new() { Id = 3, Name = "Watalappan",
                    Price = 350m, Category = "Dessert" },
        };
        _nextId = 4;
    }

    private static MenuItemDto ToDto(MenuItem m) =>
        new(m.Id, m.Name, m.Price, m.Category, m.Available);

    public IEnumerable<MenuItemDto> GetAll() =>
        _items.Select(ToDto);

    public MenuItemDto? GetById(int id)
    {
        var m = _items.FirstOrDefault(x => x.Id == id);
    }
}
```

```
        return m is null ? null : ToDto(m);  
    }
```

3. Part 2 — the create, update and delete methods (add inside the same class):

src: Services/MenuService.cs (part 2)

```
public MenuItemDto Create(CreateMenuItemDto dto)  
{  
    var item = new MenuItem  
    {  
        Id = _nextId++,  
        Name = dto.Name,  
        Price = dto.Price,  
        Category = dto.Category,  
        Available = true  
    };  
    _items.Add(item);  
    return ToDto(item);  
}  
  
public bool Update(int id, CreateMenuItemDto dto)  
{  
    var m = _items.FirstOrDefault(x => x.Id == id);  
    if (m is null) return false; // guard clause  
    m.Name = dto.Name; // replaces the editable fields  
    m.Price = dto.Price;  
    m.Category = dto.Category;  
    return true;  
}  
  
public bool Delete(int id)  
{  
    var m = _items.FirstOrDefault(x => x.Id == id);  
    if (m is null) return false;  
    _items.Remove(m);  
    return true;  
}  
}
```

4. Register the service in Program.cs (above builder.Build()):

src: Program.cs

```
builder.Services.AddSingleton<IMenuService, MenuService>();
```

WHY SINGLETON HERE?

The lecture uses `AddScoped` for services backed by a database. Here the service *is* the data store, so it must keep its list across requests — that needs `AddSingleton` (one instance for the whole app). With a real database and repository next lecture, you would switch back to `AddScoped`.

EXPECTED OUTPUT

The service compiles and is registered for dependency injection. It holds three seeded dishes in memory and exposes read and write methods. A screenshot of the registration line and a brief note on the lifetime choice.

TASK 04 • 25 minutes • LO2, LO3**Controller with GET + POST (Tested in Swagger)**

Create a thin controller that receives HTTP requests, calls the service, and returns the correct status codes. Use attribute routing and constructor injection exactly as in the lecture.

Step-by-step instructions

1. Create `Controllers/MenuController.cs` with the read endpoints and create endpoint:

```
src: Controllers/MenuController.cs
using CampusEats.Api.Dtos;
using CampusEats.Api.Services;
using Microsoft.AspNetCore.Mvc;

namespace CampusEats.Api.Controllers;

[ApiController] // auto 400 on invalid body
[Route("api/[controller]")] // → /api/menu
public class MenuController : ControllerBase
{
    private readonly IMenuService _svc;
    public MenuController(IMenuService svc)
        => _svc = svc; // injected by DI

    [HttpGet] // GET /api/menu
    public ActionResult<IEnumerable<MenuItemDto>> GetAll()
        => Ok(_svc.GetAll());

    [HttpGet("{id}")] // GET /api/menu/1
    public ActionResult<MenuItemDto> GetById(int id)
    {
        var item = _svc.GetById(id);
        return item is null ? NotFound() : Ok(item);
    }

    [HttpPost] // POST /api/menu
    public ActionResult<MenuItemDto> Create(
        [FromBody] CreateMenuItemDto dto)
    {
        var created = _svc.Create(dto);
        return CreatedAtAction(nameof(GetById),
            new { id = created.Id }, created); // 201
    }
}
```

2. Run the API and open Swagger. Test `GET /api/menu` (expect 200 and the three dishes).
3. Test `GET /api/menu/1` (200) and `GET /api/menu/99` (expect 404 Not Found).
4. Test `POST /api/menu` with this request body and confirm a 201 Created response:

POST /api/menu — request body (JSON)

```
{
  "name": "Iced Milo",
  "price": 300,
  "category": "Drinks"
}
```

5. Note the response JSON is camelCase and includes the server-assigned id:

201 Created — response body (JSON)

```
{
  "id": 4,
  "name": "Iced Milo",
  "price": 300,
  "category": "Drinks",
  "available": true
}
```

EXPECTED OUTPUT

Working GET (all and by id) and POST endpoints. Screenshots from Swagger showing: the list with 200, a 404 for a missing id, and a successful POST returning 201 with the new dish.

TASK 05 • 20 minutes • LO2, LO3, LO4**Add PUT + DELETE, Test Full CRUD & Justify (LO4)**

Complete the CRUD set with update and delete, test the whole API, then justify your REST design decisions using the lecture's method and status-code rules.

Part A — Add PUT and DELETE

1. Add these two actions inside `MenuController` (before the closing brace):

```
src: Controllers/MenuController.cs (add)

[HttpPut("{id}")]                // PUT /api/menu/1
public IActionResult Update(int id,
    [FromBody] CreateMenuItemDto dto)
{
    var ok = _svc.Update(id, dto);
    return ok ? NoContent() : NotFound();    // 204/404
}

[HttpDelete("{id}")]            // DELETE /api/menu/1
public IActionResult Delete(int id)
{
    var ok = _svc.Delete(id);
    return ok ? NoContent() : NotFound();    // 204/404
}
```

2. In Swagger, run the full sequence: `POST` a new dish, `GET` the list, `PUT` to change it (expect 204), `DELETE` it (expect 204), then `GET` the deleted id (expect 404).
3. Also test validation: `POST` with an empty name and confirm `[ApiController]` returns 400 `BadRequest` automatically.

Part B — Justify your design (LO4)

Using the lecture's HTTP-method and status-code rules, complete the table for the CampusEats Menu API:

Operation	Method	Path	Success status + why
List all dishes			
Get one dish by id			
Add a new dish			
Replace a dish			
Remove a dish			

Then write 2–3 sentences answering: why is ASP.NET Core a suitable framework for this backend? Cite at least two reasons from the lecture (e.g. performance, cross-platform/container support, batteries-included, enterprise/LTS).

EXPECTED OUTPUT

A full CRUD API (GET, POST, PUT, DELETE) tested end-to-end in Swagger with screenshots of the 201, 204, 404 and 400 responses; a completed REST-design table; and a short ASP.NET Core justification.

QUICK TEST • 10 minutes**Knowledge Check**

Answer all questions in your Word document. Write the question number and your answer. Q1–Q5 are multiple choice; Q6 is a REST-design question; Q7 is code-based; Q8 is scenario-based.

Multiple choice (choose one)

Q1. Which C# type stores money exactly?

- (a) double
- (b) float
- (c) decimal
- (d) int

Q2. What does the [ApiController] attribute give you automatically?

- (a) a database
- (b) 400 Bad Request on an invalid model
- (c) HTTPS
- (d) JWT auth

Q3. A controller needs an IMenuService. How does it get one?

- (a) new MenuService()
- (b) constructor injection (DI)
- (c) a global static variable
- (d) a query parameter

Q4. A POST that creates a new resource should return:

- (a) 200 OK
- (b) 201 Created
- (c) 204 No Content
- (d) 404 Not Found

Q5. Why return a DTO instead of the internal entity?

- (a) it is faster to type
- (b) to hide internal fields and decouple the API from the model
- (c) C# requires it
- (d) to avoid using JSON

REST design

Q6. For the operation “view one dish by id”, give the HTTP method, the path, the success status code, and the status code returned when the dish does not exist.

Code-based

Q7. Complete the route attribute and the not-found return in this action:

```
[Http__("{id}")]
public ActionResult<MenuItemDto> GetById(int id)
{
    var item = _svc.GetById(id);
    return item is null ? _____ : Ok(item);
}
```

Scenario-based

Q8. A teammate wants to change only the price of dish 2, but finds that `PUT /api/menu/2` with body `{ "price": 900 }` is rejected with 400 Bad Request. Explain why PUT behaves this way, and which HTTP method is designed for partial updates.

SUBMIT YOUR QUICK-TEST ANSWERS INSIDE THE SAME WORD DOCUMENT

Place your quick-test answers in a clearly labelled “Quick Test” section after your Task 01–05 answers.

Submission Instructions

Prepare a single Microsoft Word document containing all of your work for this lab, and a ZIP of your project if your lecturer requests it.

Your Word document must include:

- Student name
- IT number
- Lab practical number (Lab 03)
- Screenshots of the completed API project (the folder structure and the running app)
- Screenshots of Swagger (or Postman) testing each endpoint, showing the status codes (200, 201, 204, 404, 400)
- Source-code snippets for the important classes (MenuItem, the DTOs, IMenuService, MenuService, MenuController)
- A brief explanation for each task (2–3 sentences)
- Your Quick Test answers (Q1–Q8)

Project ZIP (if required by the lecturer):

- Zip your project folder *without* the `bin` and `obj` folders.

File naming and upload:

1. Rename the Word document using your IT number: `ITXXXXXXXX_Lab03.docx`
2. Example: `IT22123456_Lab03.docx`
3. If submitting the project, name the ZIP: `ITXXXXXXXX_Lab03_Project.zip`
4. Submit the Word document (and ZIP if required) to **CourseWeb** before the end of the 2-hour lab session.
5. **Late submissions will not be accepted** unless prior approval is given by the lecturer.

STUDENT INSTRUCTION

You must refer to the provided Lecture 03 material when completing the lab tasks. Your implementation and explanations should be based on the lecture content, class explanations, and the practical examples discussed during the lecture.

Evaluation Criteria (Marking Guide — 20 Marks)

For lecturer reference. Marks reward a working API, correct REST semantics, clean layered structure, and justified design choices.

Component	Marks	What earns full marks
Task 01 — Project & Swagger	3	Controller-based Web API runs; Swagger loads; clean folder structure created.
Task 02 — Model & DTOs	3	MenuItem uses decimal for price; record DTOs; input DTO omits Id; validation attributes present.
Task 03 — Service & DI	4	Interface + implementation with in-memory data; correct CRUD logic; registered for DI (lifetime justified).

Component	Marks	What earns full marks
Task 04 — Controller GET + POST	4	[ApiController] + attribute routing; constructor injection; Ok/NotFound/CreatedAtAction; tested in Swagger.
Task 05 — PUT + DELETE & LO4	4	Working PUT/DELETE with 204/404; full CRUD tested; REST-design table correct; ASP.NET Core justification.
Quick Test (Q1–Q8)	2	Q1–Q5 correct (c, b, b, b, b); Q6–Q8 show correct understanding.
Total	20	